

AWHDL チュートリアル

Agentic Workflow Harness Description Language v0.1

AI Conductor プロジェクト

2026 年 9 月 24 日

概要

AWHDL は、複数の AI・MCP ツール・決定的プログラム・人間を組み合わせた反復ループを、型付き・イベント駆動の設計として記述する言語です。本チュートリアルでは、実機で動作を確認した 6 つの設計例を順にたどり、**entity** / **architecture**、**device** と **route** の束縛、**signal** と **delta cycle**、タイマーと予算、フィードバックループ、並列実行とバリア、情報フロー検査までを学びます。

目次

1	はじめに	3
1.1	この文書の位置付け	3
1.2	AWHDL が解く問題	3
1.3	全体の構成	3
2	準備	4
2.1	必要なもの	4
2.2	ビルド	4
2.3	検査と実行のコマンド	4
3	最初の設計：entity と architecture	5
3.1	設計	5
3.2	実行	5
4	決定的な判定：deterministic device と if	6
4.1	入力の渡し方	7
5	delta cycle、タイマー、予算	8
5.1	delta cycle	8
5.2	タイマーと予算	8
6	フィードバックループ	9
7	並列実行とバリア	10

8	情報フローと安全	12
8.1	機密区分と配置	12
8.2	さらに厳しくする	12
8.3	設計の外にある安全機構	13
9	device と route	13
9.1	束縛の規則	13
9.2	AI Conductor の route (2026 年 9 月時点)	13
9.3	呼び出しと引数	14
10	v0.1 で使えないもの	14
付録 A	文法の早見表	14
付録 B	診断コード	15
付録 C	参考文献	15

1 はじめに

1.1 この文書の位置付け

本書は非規範の入門書です。v0.1 の実装範囲は docs/PROFILE_v0.1_ja.md だけが定め、言語・実行・安全の規則は LANGUAGE_SPEC、RUNTIME_SPEC、SECURITY_SPEC、IR_SPEC が定めます。本書の記述がそれらと食い違う場合は、規範文書が優先します。

本書の例はすべて examples/tutorial/ にあり、2026 年 9 月 24 日に次の 2 つで確認しました。

- aic check による構文・構造・情報フローの静的検査
- aiconductor run-design による実機での実行（27B モデル、Prolog、Lean、KDB を使用）

1.2 AWHDL が解く問題

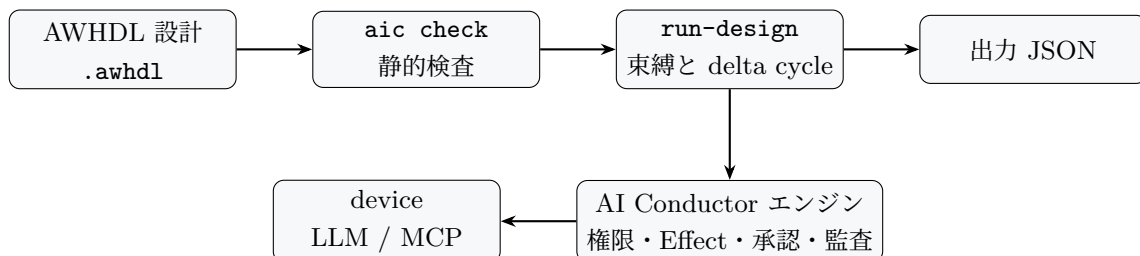
AI にツールを使わせる仕組みでは、「どの AI がどのデータを受け取るか」「いつ再実行し、いつ止めるか」「何をもって完了とするか」がプロンプトの中に埋もれがちです。AWHDL はこれらを設計として書き出し、実行前に検査できるようにします。

AWHDL の基本的な考え方

- AI やツールは **device**（入出力を持つ部品）として扱う。
- データや状態は **signal** が運ぶ。
- **process** はイベントに反応して動く。
- 実行を続けてよいか、外部へ送ってよいか、完了したかは、AI ではなく **AI Conductor ランタイム**が宣言された規則に従って決める。

構文は VHDL に似ていますが、AWHDL はハードウェアを記述する言語ではありません。名前の H は Harness（複数の実行主体を一つの制御構造に束ねるもの）を表します。

1.3 全体の構成



run-design は設計を検査・束縛したうえで delta cycle として実行します。device への呼び出しは、通常の実行と同じエンジンを通るため、権限（capability）の検査、Effect の記録、承認、監査ログがすべて適用されます。

2 準備

2.1 必要なもの

- Rust (2024 edition) と Cargo
- awhdl リポジトリ (本書の例とランタイムを含む)
- 実行 (run-design) には、AI Conductor のプロジェクトルートと、そこで設定された route (モデルや MCP サーバー)

静的検査 (aic check) だけなら、モデルや MCP サーバーは不要です。

2.2 ビルド

```
cd awhdl
cargo build -p conductor-cli -p aiconductor
# aic          : 構文と静的検査
# aiconductor : AI Conductor ランタイム (run-design を含む)
```

AI Conductor のワークスペースでは、ビルド成果物を OneDrive の外に置くため、次の設定を使います。

```
export CARGO_TARGET_DIR=~/.cache/aiconductor/cargo-target
```

以下では実行ファイルを aic、aiconductor と略記します。

2.3 検査と実行のコマンド

```
# 静的検査 (終了状態 0 = 合格)
aic check examples/tutorial/01_hello.awhdl
aic check examples/tutorial/01_hello.awhdl --output json

# 実行 (--project-root は AI Conductor のルート)
aiconductor --project-root <AIConductor> run-design \
  awhdl/examples/tutorial/01_hello.awhdl --input 'task=...'
```

--input 名前=値 は入力ポートに値を与えます。値が JSON として解釈できれば JSON、そうでなければ文字列になります。複数のポートには --input を繰り返します。

表 1 aic check の終了状態

状態	意味
0	実装済みの検査に合格
1	ソースファイルを読み取れない
2	構文解析に失敗
3	構造または情報フローの検査に失敗

3 最初の設計：entity と architecture

3.1 設計

Listing 1 examples/tutorial/01_hello.awhdl

```
1  -- 01: one entity, one architecture, one device, one process.
2  entity hello is
3      port (
4          task    : in  text<internal>;
5          answer  : out text<internal>
6      );
7  end hello;
8
9  architecture behavioral of hello is
10     -- An agent device bound to the configured "initial_coding" model route.
11     device coder : agent generic (location => local, route => "initial_coding");
12 begin
13     process(task)
14     begin
15         coder.run(task) -> answer;
16     end process;
17 end behavioral;
```

entity 設計の外部インターフェースです。port に入力 (in) と出力 (out) を並べます。

型 text<internal> 型名の後の <...> は機密区分です。public、internal、restricted の 3 段階があり、第 8 節の情報フロー検査に使われます。

architecture entity の中身です。is と begin の間に宣言を、begin と end の間に process を書きます。

device AI やツールを表す部品です。種類は agent、mcp、deterministic の 3 つです。generic で、どの **route** (AI Conductor の設定にある実行経路) に結び付けるか、どこで動くか (location) を宣言します。

process(task) 括弧内は感度リストです。task が変化すると本体が実行されます。

coder.run(task) -> answer; device を呼び出し、結果を answer に書き込みます。agent device の場合、メソッド名 (ここでは run) は結果に影響しません。

3.2 実行

```
aiconductor --project-root . run-design awhdl/examples/tutorial/01_hello.awhdl \
--input 'task=Rustで2つのi32を安全に足す関数を1つだけ書いてください。説明は1行で.'
```

Listing 2 実行結果 (27B モデル、約 4 秒)

```
{
  "status": "quiescent",
```

```

"reason": null,
"outputs": {
  "answer": "```\nfn safe_add(a: i32, b: i32) -> Option<i32> {\n    a.checked_add(\n    b)\n}\n```\n\n`checked_add` はオーバーフロー時に `None` を返すため、..."
},
"delta_cycles": 2,
"device_calls": 1
}

```

`status quiescent` は、処理すべきイベントがなくなって自然に終わったことを表します。予算で打ち切られた場合は `exhausted` になり、`reason` に理由が入ります。

`outputs` 出力ポートの最終値です。

`delta_cycles` 実行した `delta cycle` の数です（第 5 節）。

`device_calls` `device` を呼び出した回数です。

4 決定的な判定：deterministic device と if

モデルの出力をそのまま「合格」とみなすと、モデルの誤りがそのまま結論になります。AWHDL では、ツールが返した構造化データを使って、判定を設計の中で決定的に行えます。

Listing 3 examples/tutorial/02_prolog_verdict.awhdl

```

1  -- 02: a deterministic device and a deterministic verdict on its result.
2  entity prolog_verdict is
3      port (
4          task      : in  text<internal>;
5          verdict   : out text<internal>
6      );
7  end prolog_verdict;
8
9  architecture checked of prolog_verdict is
10     device rules : deterministic generic (location => local, route => "logic_rules");
11     signal result : data<internal>;
12     begin
13         process(task)
14         begin
15             rules.run_prolog(task) timeout 1 min -> result;
16         end process;
17
18     -- The verdict is computed by AWHDL, not by a model.
19     process(result)
20     begin
21         if result.structured.solution_count > 0 then
22             verdict <= "holds";
23         else
24             verdict <= "does not hold";
25         end if;
26     end process;

```

```

27
28     process(rules.failed)
29     begin
30         verdict <= "prolog rejected the input";
31     end process;
32 end checked;

```

deterministic 読み取り専用の MCP route にだけ結び付けられる device です。書き込みを伴う route (MATLAB や KDB など) には **mcp** を使います。

signal result : data<internal>; 設計内部の信号です。MCP device の結果は JSON オブジェクトで、**result.text** (ツールの出力文字列) や **result.structured** (ツールが返した JSON) としてフィールドを読めます。

timeout 1 min 呼び出しの制限時間です。時間の単位は **ms**、**sec**、**min**、**hour**、**day** です。

if ... elsif ... else ... end if; 条件分岐です。

rules.failed 呼び出しが失敗したときに起きるイベントです。成功時は **rules.done**、制限時間の超過時は **rules.timeout** と **rules.failed** が起きます。

4.1 入力の渡し方

v0.1 の式にはオブジェクトを組み立てる構文がないため、device に渡すのは主に 1 つの文字列です。ツールの引数 (Prolog のプログラムと問い合わせなど) は、ランタイムのアダプタが入力文字列から取り出します。Prolog では、バッククォートで囲んだ節の並びをプログラム、最後の単一のゴールを問い合わせとして扱います。

```

aiconductor --project-root . run-design awhdl/examples/tutorial/02_prolog_verdict.awhdl
\
--input 'task=`parent(tom,bob). parent(bob,ann). grandparent(X,Z) :- parent(X,Y),
parent(Y,Z).` `grandparent(tom,ann).`'
# => "verdict": "holds"          (delta_cycles 3, device_calls 1)

aiconductor --project-root . run-design awhdl/examples/tutorial/02_prolog_verdict.awhdl
\
--input 'task=`parent(tom,bob). grandparent(X,Z) :- parent(X,Y), parent(Y,Z).` `
grandparent(tom,ann).`'
# => "verdict": "does not hold"

```

ポイント

「成り立つかどうか」は Prolog が返した **solution_count** で決まり、モデルは関与しません。モデルに判断させるのは、決定的に判定できない部分だけにするのが AWHDL での基本的な設計です。

5 delta cycle、タイマー、予算

5.1 delta cycle

AWHDL の実行は **delta cycle** の繰り返しです。1 つの delta cycle では次の順に処理します。

1. 直前に起きたイベントに感度を持つ process を選ぶ。
2. 選んだ process をすべて、同じ時点の値のスナップショットに対して実行する。process 内の代入 ($\leq$) や device の結果は、すぐには反映されない。
3. delta cycle の終わりに、書き込みをまとめて反映する。値が**実際に変わった** signal だけが次のイベント（名前 と 名前.changed）を起こす。
4. 同じ delta cycle で 2 つの process が同じ signal に異なる値を書くとエラーになる。
5. `assert always` / `assert never` を評価する。

イベントが残らず、タイマーもなければ、実行は `quiescent` で終わります。

5.2 タイマーと予算

Listing 4 examples/tutorial/03_timer_budget.awhdl

```
1  -- 03: delta cycles driven by a timer, bounded by a budget. No devices.
2  entity ticker is
3      port (
4          ticks : out data<internal>
5      );
6  end ticker;
7
8  architecture counting of ticker is
9      timer tick : period 1 sec;
10     signal count : data<internal> := 0;
11     budget run_budget is
12         wall_time <= 3 sec;
13     end budget;
14 begin
15     process(tick)
16     begin
17         count <= count + 1;
18     end process;
19
20     process(count)
21     begin
22         ticks <= count;
23     end process;
24
25     assert always (count <= 10);
26 end counting;
```

timer tick : period 1 sec; 1 秒ごとに tick イベントを起こします。タイマーを使う設計には、iterations か wall_time の予算が必須です。

budget 実行の上限です。v0.1 で使える項目は iterations (delta cycle 数)、wall_time (経過時間)、tool_calls、model_calls です。

signal count : data<internal> := 0; 初期値付きの signal です。初期値は定数に限ります。

assert always (count <= 10); すべての delta cycle の終わりに成り立つべき条件です。破られると実行は失敗します。

Listing 5 実行結果 (デバイスなし、約 3 秒)

```
{ "status": "exhausted", "reason": "wall_time budget",
  "outputs": { "ticks": 2 }, "delta_cycles": 6, "device_calls": 0 }
```

設計の予算とランタイムの予算

設計の budget は、AI Conductor の config/runtime.toml ([loop] 節) の上限をさらに狭めるだけで、広げることはできません。例えばランタイムの max_iterations が 8 のとき、設計で iterations <= 20 と書いても 8 delta cycle で打ち切られます。タイマー間隔を 500ms にすると、この例は設計の wall_time より先にランタイムの上限 (iteration budget exhausted) で止まります。

6 フィードバックループ

検証して、結果によって再実行する。AWHDL では、このループをイベントの連鎖として書きます。

Listing 6 examples/tutorial/04_feedback_loop.awhdl

```
1  -- 04: a feedback loop that repeats a device call until a condition holds.
2  entity kdb_loop is
3      port (
4          code    : in  text<internal>;
5          rounds  : out data<internal>;
6          last    : out data<internal>
7      );
8  end kdb_loop;
9
10 architecture feedback of kdb_loop is
11     device q : mcp generic (location => local, route => "table_analysis");
12     signal round : data<internal> := 0;
13     signal result : data<internal>;
14     budget loop_budget is
15         iterations <= 20;
16         tool_calls <= 5;
17     end budget;
18 begin
19     process(code)
20     begin
```

```

21     round <= 1;
22 end process;
23
24 -- Each change of round runs one more call, until three rounds are done.
25 process(round)
26 begin
27     if round >= 1 and round <= 3 then
28         q.run_q(code) timeout 1 min -> result;
29     end if;
30 end process;
31
32 -- q.done fires after every successful call, even when the result is
33 -- unchanged; a signal event would fire only when its value changes.
34 process(q.done)
35 begin
36     last <= result.text;
37     rounds <= round;
38     round <= round + 1;
39 end process;
40
41 assert never (round > 4);
42 end feedback;

```

1. code が与えられると round <= 1。
2. round が変わるたびに、条件を満たせば KDB/q を呼ぶ。
3. 呼び出しが成功すると q.done が起き、round を 1 増やす。
4. round が 4 になると呼び出さなくなり、イベントが尽きて終わる。

signal のイベントと device のイベント

signal のイベントは値が**変わったとき**だけ起きます。同じ計算を繰り返すと結果が変わらないため、process(result) ではループが 1 回で止まってしまいます。呼び出しの完了ごとに必ず起きる q.done を使うのはこのためです。

```

aiconductor --project-root . run-design awhdl/examples/tutorial/04_feedback_loop.awhdl \
--input 'code=sum til 101'
# => "rounds": 3, "last": "{ \"ok\": true, ... \"stdout\": \"5050\\n\" ... }"
#   status quiescent, delta_cycles 8, device_calls 3

```

tool_calls <= 5 は、万一条件を書き誤っても呼び出しが 5 回を超えないための安全策です。assert never (round > 4); は「このループは 4 回目以降に進まない」という設計者の意図を、実行時に検査できる形で残します。

7 並列実行とバリア

独立した検査は同時に始め、両方がそろった時点で次へ進みたいことがあります。

Listing 7 examples/tutorial/05_parallel_barrier.awhdl

```

1  -- 05: two checks started together; a barrier fires when both have results.
2  entity two_checks is
3      port (
4          rules_task : in  text<internal>;
5          proof_task : in  text<internal>;
6          summary    : out text<internal>
7      );
8  end two_checks;
9
10 architecture joined of two_checks is
11     device rules : deterministic generic (location => local, route => "logic_rules");
12     device prover : deterministic generic (location => local, route => "logic_proof"
13     );
14     signal rule_result, proof_result : data<internal>;
15     barrier both (rule_result, proof_result);
16 begin
17     process(proof_task)
18     begin
19         parallel
20             rules.run_prolog(rules_task) -> rule_result;
21             prover.check_lean_code(proof_task) -> proof_result;
22         end parallel;
23     end process;
24
25     process(both.ready)
26     begin
27         summary <= "rules and proof both returned";
28     end process;
29
30     process(prover.failed)
31     begin
32         summary <= "the proof was rejected";
33     end process;
34 end joined;

```

parallel ... end parallel; 中の呼び出しは論理的に並列で、結果は同じ delta cycle の終わりにまとめて反映されます (v0.1 のランタイムは実際には 1 つずつ順に呼び出します)。2 つの分岐が同じ signal に書き込むことはできません (E215)。

barrier both (rule_result, proof_result); 列挙した signal がすべて書き込まれると both.ready が起きます。

```

aiconductor --project-root . run-design awhdl/examples/tutorial/05_parallel_barrier.
awhdl \
--input 'rules_task=`edge(a,b). path(X,Y) :- edge(X,Y).` `path(a,b).`' \
--input 'proof_task=theorem two : 1 + 1 = 2 := by rfl'
# => "summary": "rules and proof both returned"    (device_calls 2)

```

```
# 誤った定理を与えると Lean が拒否し、proof_result は書き込まれない
--input 'proof_task=theorem bad : 2 + 2 = 5 := by rfl'
# => "summary": "the proof was rejected"
```

失敗した呼び出しは出力 signal に書き込まないため、バリアは成立しません。失敗は `prover.failed` で別に扱います。

8 情報フローと安全

8.1 機密区分と配置

signal とポートの型には機密区分 (`public < internal < restricted`) を付けます。区分のない型は `restricted` として扱われます。device は location (`local` または `cloud`) を宣言し、route の実際の配置と一致しなければ束縛に失敗します。

Listing 8 examples/tutorial/06_information_flow.awhdl

```
1  -- 06: restricted data must never reach a cloud device. aic check rejects this.
2  entity leak is
3    port (
4      secret_notes : in  text<restricted>;
5      advice       : out text<restricted>
6    );
7  end leak;
8
9  architecture unsafe of leak is
10     device reviewer : agent generic (location => cloud, route => "deep_reasoning");
11  begin
12     process(secret_notes)
13     begin
14         reviewer.codex(secret_notes) -> advice;
15     end process;
16  end unsafe;
```

```
$ aic check awhdl/examples/tutorial/06_information_flow.awhdl
awhdl/examples/tutorial/06_information_flow.awhdl:14:9: AWHDL-E301: Restricted data
    cannot flow to cloud device reviewer
$ echo $?
3
```

`restricted` のデータがクラウドの device に流れる経路は、実行前に拒否されます。ランタイムも呼び出しの直前に同じ検査を行うため、静的検査をすり抜けた場合でも送信されません。

8.2 さらに厳しくする

- `assert never (internal -> cloud);` を書くと、`internal` 以上のデータもクラウドへ送れなくなります。
- device に `clearance => public` などをつけると、その device が受け取れる区分の上限を決められます (超えると E302)。

- 低い区分の signal に高い区分のデータを代入することはできません (E303)。v0.1 には区分を下げる仕組み (declassifier) はありません。

8.3 設計の外にある安全機構

次の規則は設計ではなく、AI Conductor の設定とランタイムが強制します。

- **capability**：どの route がどのツール・ファイル・モデルを使えるか。設定は `config/capabilities.toml` にあります。
- **Effect と承認**：書き込みを伴う呼び出しの記録と、外部への書き込み前の人間の承認。
- **監査**：実行ごとの `.runtime/runs/<id>/events.jsonl`。

設計から device を呼び出すと、これらがすべて適用されます。設計の側からこれらを無効にする構文はありません。

9 device と route

9.1 束縛の規則

表 2 device の種類と結び付けられる route

device の種類	結び付けられる route	例
agent	モデルの route、または Codex	initial_coding、deep_reasoning
mcp	MCP の route (Codex 以外)	calculation_graphing、table_analysis
deterministic	読み取り専用の MCP の route	logic_rules、logic_proof、text_filtering

主制御 (planner) の route (loop) には結び付けられません。

9.2 AI Conductor の route (2026 年 9 月時点)

route	種類	実行先
initial_coding	モデル	192.168.6.10 の RTX 4090 (Qwen3.8 27B)
japanese_polishing	モデル	192.168.6.10 (llm-jp-4 8B)
vision	モデル	192.168.6.20 の RX 9060 XT (Qwen3-VL 8B)
logic_rules	MCP (読み取り)	192.168.6.10 の SWI-Prolog
logic_proof	MCP (読み取り)	192.168.6.10 の Lean 4.33.1
text_filtering	MCP (読み取り)	ローカルの制限付き Filter
table_analysis	MCP (書き込み)	192.168.6.10 の KDB/q
calculation_graphing	MCP (書き込み)	192.168.6.10 の MATLAB R2026a
deep_reasoning	Codex (cloud)	Codex CLI (読み取り専用)
open_data_acquisition	Codex (cloud)	Codex CLI (承認が必要な外部書き込み)

9.3 呼び出しと引数

- 引数が 1 つの文字列なら、それが device への入力になります。
- MCP の route では、メソッド名がその route のツール名と一致すれば、そのツールを使います（例：rules.run_prolog、prover.check_lean_code、q.run_q）。
- ツールの引数はアダプタが入力から取り出します。Prolog はバッククォートの節と問い合わせ、Lean は宣言（theorem など）、KDB は q のコード、Filter はプロジェクト内のパスです。
- MATLAB と KDB では、日本語の文章をコードとして送ることはせず、呼び出しを失敗させます。

10 v0.1 で使えないもの

次の構文は v0.1 に含まれず、構文エラーとして拒否されます。黙って受理されることはありません。

- case、await、type 宣言と状態機械、retry、approve、policy、export、configuration、時相アサーション
- Event 宣言と完了条件（completion when hard {...}）の構文。意味論は実装済みで、完了条件は現在 config/routing-defaults.toml で与えます。
- 機密区分 confidential / secret、配置 sandbox / private_cloud / external (E401)

付録 A 文法の早見表

```
1  entity name is
2      port ( a : in text<internal>; b : out data<public> );
3  end name;
4
5  architecture arch of name is
6      device d : agent | mcp | deterministic
7          generic (route => "<route>", location => local | cloud,
8                  clearance => public | internal | restricted);
9      signal s1, s2 : data<internal> := 0;
10     timer t : period 1 sec;
11     budget lim is iterations <= 10; wall_time <= 5 min;
12         tool_calls <= 3; model_calls <= 2; end budget;
13     barrier bar (s1, s2);
14 begin
15     assert always (<expr>);
16     assert never (<expr>);
17     assert never (internal -> cloud);
18
19     process(a, s1.changed, d.done, d.failed, d.timeout, t, bar.ready)
20         timeout 2 min;
21 begin
```

```

22     d.method(<expr>) timeout 30 sec -> s1;
23     s2 <= s1 + 1;
24     if <expr> then null; elsif <expr> then null; else null; end if;
25     parallel d.m1(a) -> s1; d.m2(a) -> s2; end parallel;
26     assert <expr>;
27     on timeout
28         b <= "timed out";
29     end process;
30 end arch;

```

式の結合は弱い順に or、and、not、比較 (=、/=、<、<=、>、>=)、整数の + / - です。比較は数値どうし、または文字列どうしで行います。

付録 B 診断コード

コード	意味
E001	ソースファイルを読み取れない
E101	構文エラー
E201–E205	重複した entity・ポート・device・宣言、未知の entity
E206	感度リストの未知の名前
E207 / E208 / E209	未知の device、未知の出力先
E210	式の中の未知の名前
E211	正でない時間（タイマー周期、制限時間）
E212	不正または重複した予算項目
E214	signal でないバリアのメンバー
E215	2 つの parallel 分岐が同じ signal に書き込む
E217	入力ポートへの書き込み
E218	device の generic の重複や型の誤り
E301	クラウドの device へ流せないデータ
E302	device の clearance を超えるデータ
E303	高い区分のデータを低い区分の signal に保存
E401	v0.1 の Profile に含まれない構成要素
E402	未知の名前（区分、配置など）

付録 C 参考文献

- awhdl/docs/PROFILE_v0.1_ja.md：v0.1 の範囲と実装状況（正本）
- awhdl/docs/LANGUAGE_SPEC_ja.md、AWHDL_LANGUAGE_SPEC_v0.2_ja.md：言語仕様
- awhdl/docs/RUNTIME_SPEC_ja.md：delta cycle、Effect、完了評価
- awhdl/docs/SECURITY_SPEC_ja.md：機密区分、capability、承認
- awhdl/docs/GETTING_STARTED_ja.md：最初の一步